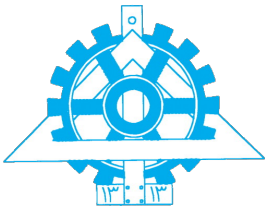
University of Tehran

College of Engineering

Department of Electrical and Computer Engineering



Introduction to Data Science

Assignment 1

Instructors: Dr. Bahrak, Dr. Yaghoobzadeh

Prepared by:

Parsa Saednia (810102460)

Mohammad Hossein Mazaheri (810102585)
Mohammad Reza Pirhadi (810102423)
Student No: Group Submission

May 18, 2026

Contents

List of Figures

Introduction

This report outlines the findings and visualizations for Data Science Assignment 1. The project encompasses three primary practical tasks: statistical sampling and inference using 2D Gaussian distributions, storytelling through interactive Tableau dashboards for Global Tuberculosis data, and the visual improvement of a financial Power BI dashboard. The programmatic implementation for data processing and mathematical modeling can be found in the accompanying Jupyter Notebook.

Task 1: Sampling and Inference

2.1 1. Define and Visualize the Distribution

Figure 1 displays the probability density function (PDF) of a standard 2D Gaussian distribution centered at the origin (0,0). The color intensity represents the probability density, with the darkest red region indicating the highest probability at the mean. The color fades outward in a perfectly uniform gradient as the probability density approaches zero at the tails of the distribution.

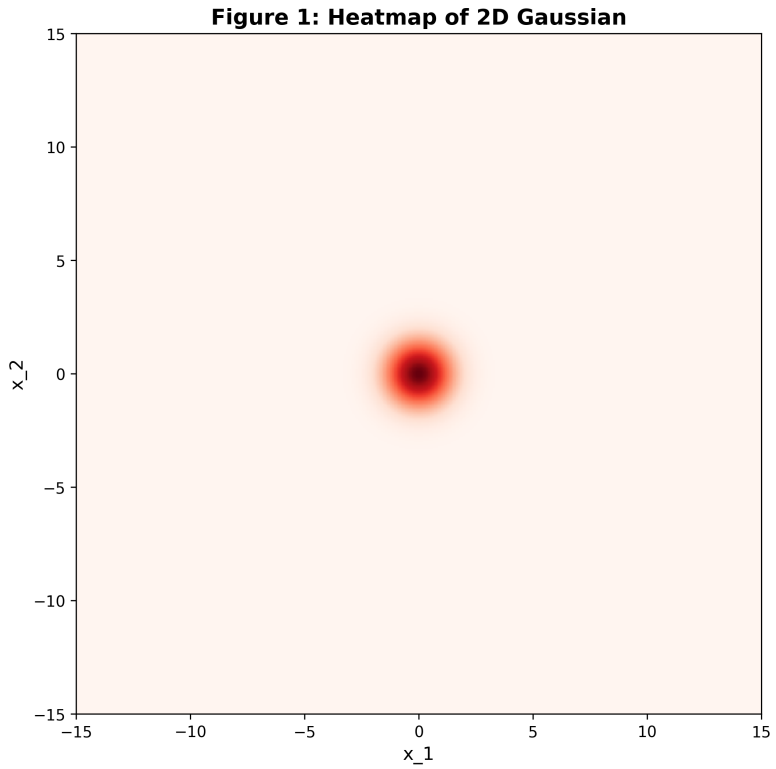


Figure 1: Heatmap of the 2D Gaussian target distribution ($\mu = [0, 0]$, $\Sigma = I$)

Theoretical Questions

1. Why is the distribution symmetric?

The distribution is symmetric because the covariance matrix is the Identity matrix (I). This means the variance along both the x_1 and x_2 axes is identically equal to 1, and the variables are completely independent (their covariance is 0). Because the spread is equal in all directions and there is no correlation skewing the data, the probability density depends purely on the Euclidean distance from the mean, resulting in perfect circular (radial) symmetry.

2. What role does the covariance matrix play in the shape of the distribution?

The covariance matrix completely dictates the spatial spread, shape, and orientation of the Gaussian distribution:

- The **diagonal elements (variances)** control the width or spread of the distribution along each individual axis.
- The **off-diagonal elements (covariances)** control the linear correlation between the variables, which determines the ”tilt” or rotation of the distribution.

In our specific case, because the off-diagonal elements are exactly zero and the diagonal elements are equal, the distribution has no tilt and forms a perfect circle rather than an elongated

ellipse.

2.2 2. Sampling Methods

In this section, we generate 1000 samples from the target distribution using two distinct methods: direct sampling via NumPy's built-in multivariate normal generator, and Langevin Dynamics sampling.

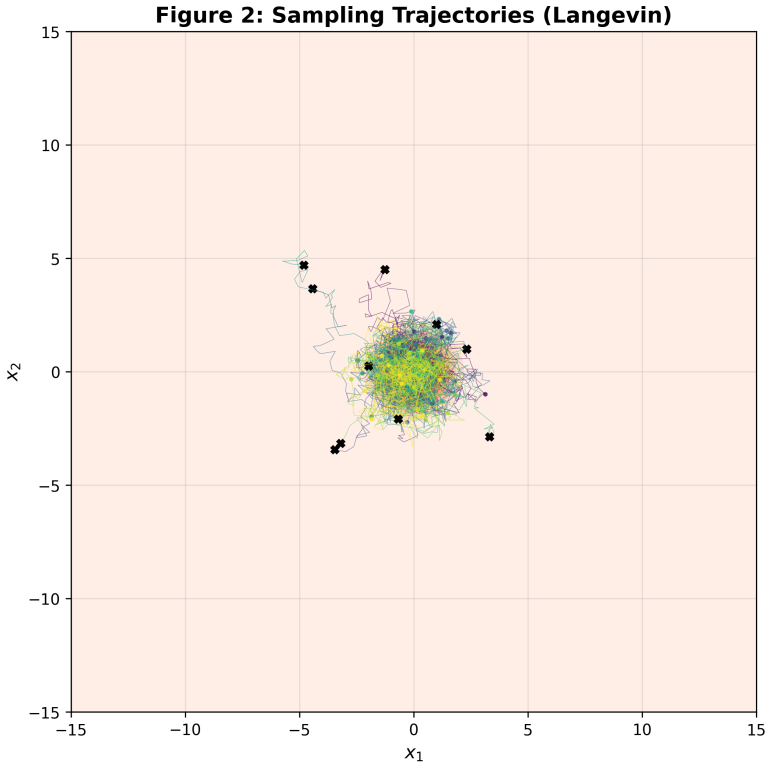


Figure 2: Sampling Trajectories of 10 particles using Langevin Dynamics

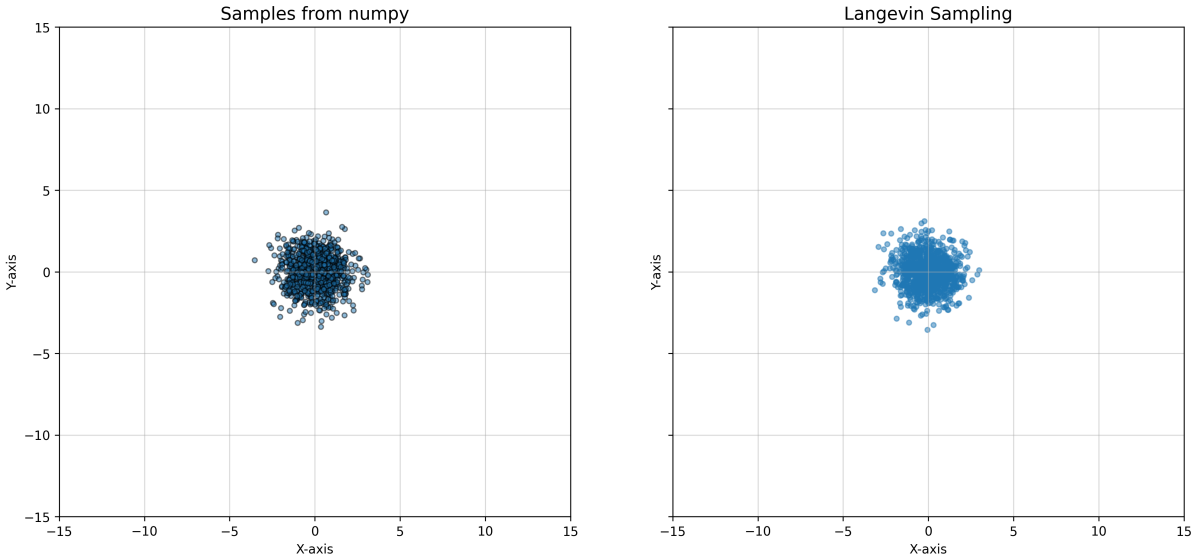


Figure 3: Comparison of Final Sampled Points: Direct Sampling (left) vs. Langevin Dynamics (right)

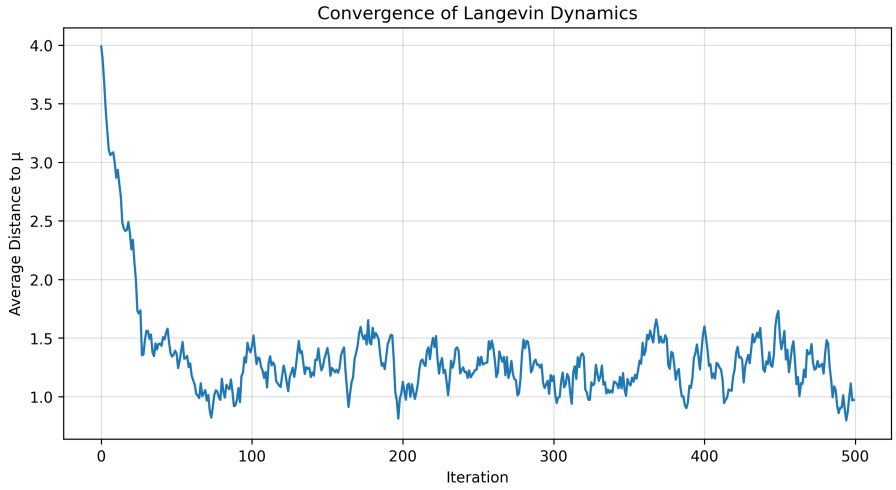


Figure 4: Convergence Analysis: Average distance of samples from the origin over iterations

Questions

1. How does changing the step size affect convergence speed and stability of the sampler?

The step size (ϵ) controls the balance between speed and stability. A larger step size allows particles to take larger strides toward the high-density regions, converging faster, but it risks overshooting the target distribution, leading to instability. Conversely, a smaller step size ensures stability but requires significantly more iterations to reach the stationary distribution.

2. How many iterations are required for the sampler to reach a stable distribution for your chosen step size? Explain how you determined this.

Based on the convergence plot (Figure 4), the sampler reaches a stable distribution after approximately 30 iterations. This was determined by observing the average distance of the particles from the origin. The distance drops rapidly during the initial phase and then levels off around iteration 30, where it begins to steadily oscillate.

3. Compare the scatter plot with the density contours and explain why more samples appear near the origin.

The density contours represent the probability density function, which peaks at the origin (0,0) for our standard Gaussian. The samples cluster densely around this central point because the score function ($\nabla_x \log p(x)$) actively directs the particles toward regions of higher probability, pulling them inward from the edges.

4. If you remove the noise term from the update rule, what behavior do you expect and why?

If the random noise term ($\sqrt{2\epsilon}z_t$) is removed, the update rule becomes purely deterministic, effectively turning into gradient ascent. Without the noise to disperse the particles, every particle would climb the probability gradient and permanently collapse exactly onto the origin (the mode). It would find the maximum probability point but fail to sample the variance of the distribution.

5. Do all trajectories converge to similar regions of the space? What does this indicate about the sampler?

Yes, as shown in the trajectory plot, all particle trajectories converge toward the central region around the origin, but they continue to fluctuate within that area rather than converging to a single point. This indicates that the sampler is working correctly: it successfully navigates to the high-probability region while maintaining the natural variance expected from the target Gaussian distribution.

2.3 3. Inference from Samples

Based on the programmatic evaluation, we calculated the estimated means and variances for both sampling methods and compared them against the true target parameters ($\mu = [0, 0]$, Var $= [1, 1]$). The results of the 1000 generated samples are summarized in Table 1.

Metric	NumPy Direct Sampling	Langevin Dynamics
Estimated Mean (x_1, x_2)	$[0.0548, 0.0109]$	$[-0.0582, -0.0358]$
Estimated Variance (x_1, x_2)	$[1.0085, 1.0462]$	$[0.9567, 0.9897]$
Absolute Mean Error	$[0.0548, 0.0109]$	$[0.0582, 0.0358]$
Absolute Variance Error	$[0.0085, 0.0462]$	$[0.0433, 0.0103]$

Table 1: Inference parameters and absolute errors for both sampling methods.

As seen in the results, Direct Sampling produced very low absolute errors because it draws exact, independent samples. Langevin Dynamics also approximated the parameters well, but showed slightly higher errors. This happens because the computer simulation must estimate the continuous paths using discrete, straight-line jumps, which introduces a small mathematical bias.

Questions

1. Why can the Langevin estimates still be biased even when the final samples come from independent chains?

The bias in Langevin dynamics is fundamentally caused by the step size (discretization error), not by a lack of samples. In a perfect mathematical model, the particles move in perfectly smooth, continuous curves. However, the computer forces the particles to take distinct, straight-line jumps. Because these jumps are straight, they slightly overshoot the true, curved shape of the distribution. Even if we run multiple completely independent chains (particles) to reduce random noise, every single chain is still forced to use this same flawed ”straight-line” approximation. Therefore, their final results will always average out to a slightly distorted distribution.

2. How do the number of iterations T and the step size ϵ affect the accuracy of the Langevin samples?

The accuracy relies on a trade-off between these two parameters:

- **Number of iterations (T):** This determines how long the simulation runs. A larger T is necessary to give the particles enough time to move away from their random starting locations (overcoming ”burn-in”) and fully explore the target distribution.
- **Step size (ϵ):** This controls the physical length of each jump. A smaller ϵ makes the straight-line jumps tiny, which allows them to trace the true curves of the distribution

much more accurately (lower bias). However, taking tiny steps means the particles move very slowly across the space, so you are forced to use a much larger T to give them enough time to explore. Conversely, a larger step size lets them explore faster, but increases the error because large jumps heavily overshoot the curves.
